COMP 441 — Software Analysis and Testing

Week 12 — CI/CD Pipeline, Quality Gates and Final Evidence

Group A · Track D · TOML Kit

Period	Week 12
Evidence folder	reports/week12
Repository	github.com/the-dezeray/tomlkit
Commit(s)	30ad2f0 / 524355a
Generated	2026-10-27
Contents	6 sections

This pack collects the week's human-readable report and key evidence. Raw AI-only logs are listed in the inventory and intentionally not printed.

Contents

Week 12 CI/CD evidence snapshot . . . 3
- Green run . . . 3
- Headline metrics . . . 3
- Downloadable artifacts . . . 3
- What the pipeline proves . . . 3

COMP 441 CI/CD . . . 4
- First-time setup on the fork . . . 4
- Trigger map . . . 4
- Job graph . . . 4
- Jobs and commands . . . 5
- Downloadable artifacts . . . 5
- Quality gates . . . 5
- Current evidence (Week 11 v2) . . . 6
- Week 7 risk-based test order . . . 6
- Reproducibility . . . 6
- Security evidence . . . 6
- Branch protection / ruleset . . . 6
- Mapping to the COMP 441 brief . . . 7
- Retired workflow . . . 7

10-minute live CI/CD demonstration . . . 7
- Before the session . . . 7
- Timeline . . . 8
- Fallback if the live run is unavailable . . . 10
- CI workflow definition . . . 10
- Consolidated defect log (final) . . . 12
- Raw evidence inventory . . . 12

Source: reports\week12\ci-evidence.md

Week 12 CI/CD evidence snapshot

Group A · tomlkit · COMP 441 AI-Augmented Test Suite

This is the CI/CD evidence snapshot for the final submission. It is not a duplicate of the Week 11 mutation report: the authoritative mutation analysis is `week11/mutation_v2/mutation_v2_report.md` (PR #4). This file records the pipeline, the green run and the downloadable evidence.

Green run

Field	Value
Workflow	`COMP 441 CI`
Run	40412867639
Commit	`30ad2f0f8b05` (`master`, after PRs #4 and #5)
Result	**success**
Jobs	15 successful; mutation and performance correctly skipped on a PR run
Pipeline PR	#5
Mutation v2 PR	#4

Headline metrics

Metric	Value	Source
Tests collected	1,507	`junit.xml`
Passed / xfailed	1,502 / 5	`junit.xml`
Statement coverage	91.62%	`coverage.json`
Branch coverage	85.62%	`coverage.json`
Combined coverage	89.88% (local canonical run)	`coverage.json`
Mutation score v2	77.4%	`week11/mutation_v2/mutation_v2_summary.json`
Covered-code mutation score	81.3%	same
Mutation v1 baseline	57.8%	`week7/mutation_v1_summary.json`
RTM references	16 requirements, 35 references, 15 with tests	`rtm_validation.json`
Week 7 priority APFD	0.7722	`week7/prioritisation_eval.json`
Defect density	1.92 / KLOC	`reports/defects.csv` + radon

The combined coverage in the downloaded CI dashboard is 91.62% statement / 85.62% branch (platform-dependent rounding and Python version can move the last decimal); the local canonical run reported 89.88% combined. The gate is evaluated against explicit statement and branch thresholds, not the rounded display value.

Downloadable artifacts

From the green run:

```
gh run download 40412867639 -n dashboard
gh run download 40412867639 -n hybrid-evidence
gh run download 40412867639 -n ai-baseline-evidence
gh run download 40412867639 -n rtm-evidence
gh run download 40412867639 -n quality-evidence
gh run download 40412867639 -n security-evidence
gh run download 40412867639 -n junit-unit-ubuntu-latest-3.12
```

Mutation evidence is produced by the nightly/manual mutation job:

```
gh workflow run ci.yml --ref master -f run_mutation=full
gh run download <mutation-run-id> -n mutation-evidence
```

What the pipeline proves

- The **hybrid suite**, not only `tests/`, is the merge gate.
- The **Week 7 risk order** is executable in CI (`priority_order.json`).
- Coverage, RTM references and test failures are blocking.
- Mutation, performance, security and upstream compatibility are evidence jobs, kept off the fast PR path.
- Every report number is downloadable; the dashboard is also written to the run summary.

- The release workflow builds, checks and attaches a distribution artifact and cannot publish to PyPI from the student fork.

See `docs/ci-cd.md` for the job map, quality gates and mapping to the COMP 441 brief, and `docs/ci-cd-demo.md` for the 10-minute live demonstration.

Source: docs\ci-cd.md

COMP 441 CI/CD

This document describes the CI/CD pipeline for the tomlkit test suite project. The executable definition is `.github/workflows/ci.yml`; `tests.yml` has been retired so there is one source of truth for the project pipeline.

First-time setup on the fork

The repository is a fork of `python-poetry/tomlkit`. GitHub Actions was enabled for the fork, but the workflow files had never executed (`gh run list` returned zero runs before this change). Before the live demonstration:

1. Open **Actions** in `the-dezeray/tomlkit`.
2. If GitHub shows the fork banner, choose **I understand my workflows, go ahead and enable them**.
3. Run **COMP 441 CI** → **Run workflow** once with `run_mutation = none`.
4. Confirm a green run exists and download one artifact as a smoke test.

A workflow-dispatch run is used so the first demonstration does not need a new commit.

Trigger map

Event	What runs	Why
`pull_request`	quality, unit matrix, AI baseline, hybrid + coverage, RTM, security, dashboard	Fast, reviewable evidence for every change
`push` to `master`	same as PR	Protects the integration branch
`workflow_dispatch`	all fast jobs; optional full mutation and performance	Live demonstration and on-demand evidence
`schedule` (Monday 03:00 UTC)	all fast jobs, full mutation, performance, upstream compatibility	Expensive and external checks run off the critical path

Concurrency cancels superseded PR runs. Permissions are read-only except for the release workflow.

Job graph

```
flowchart LR
  Q[quality] --> G{required checks}
  U[unit matrix] --> G
  A[AI baseline] --> G
  H[hybrid + coverage] --> C[coverage gate]
  C --> G
  R[RTM validation] --> G
  S[security evidence] --> G
  H --> D[dashboard]
  R --> D
  M[mutation / nightly] -.-> D
  P[performance / nightly] -.-> D
  G --> B[merge]
```

`quality`, `unit`, `hybrid`/coverage, and RTM are the intended required checks. Mutation, performance, security evidence, and upstream compatibility are nightly or non-blocking so a long external run does not block a small PR.

Jobs and commands

Job	Core command	Output
quality	ruff check, ruff format --check, mypy tomlkit	Blocking for ci/ scripts; project baseline reports are evidence-only
unit	pytest -q tests on Ubuntu/Windows/macOS × Python 3.9/3.12/3.14	JUnit per matrix cell
ai-baseline	pytest -q tests_ai --cov=tomlkit --cov-branch	AI-only coverage + JUnit
hybrid	pytest -q -p ci.priority_plugin tests tests_ai tests_manual tests_integration tests_bdd tests_property tests_tdd --cov=tomlkit --cov-branch	Coverage XML/JSON/HTML, JUnit, priority report
rtm	pytest --collect-only -q then python ci/validate_rtm.py	rtm_validation.json, rtm_validation.md
mutation	mutmut run --max-children 4 with mutmut 3.8.0	Raw results, summary JSON, survivors
performance	python docs/week9/performance_check.py	Throughput text evidence
security	pip-audit, bandit	Security JSON/text evidence
dashboard	python ci/build_dashboard.py	dashboard.md, dashboard.csv, job summary

Downloadable artifacts

Every artifact is attached to the workflow run. Reviewers and the final report do not need a third-party coverage service.

Artifact	Download with	Contents
hybrid-evidence	gh run download <run-id> -n hybrid-evidence	coverage.xml, coverage.json, coverage_gate.json, junit.xml, htmlcov/, priority_order.json
ai-baseline-evidence	gh run download <run-id> -n ai-baseline-evidence	AI-only coverage and JUnit
quality-evidence	gh run download <run-id> -n quality-evidence	Baseline ruff and mypy reports
rtm-evidence	gh run download <run-id> -n rtm-evidence	RTM validation JSON/Markdown, collected test IDs
mutation-evidence	gh run download <run-id> -n mutation-evidence	Raw mutmut results, summary JSON, survivors
performance-evidence	gh run download <run-id> -n performance-evidence	performance.txt
security-evidence	gh run download <run-id> -n security-evidence	pip-audit.json, bandit.txt
dashboard	gh run download <run-id> -n dashboard	dashboard.md, dashboard.csv
junit-unit-*	gh run download <run-id> -n junit-unit-ubuntu-latest-3.12	Unit JUnit for one matrix cell

Open reports/htmlcov/index.html locally for the line-by-line coverage view. The workflow also writes the dashboard to the run's **Summary** page.

Quality gates

The canonical coverage run is the hybrid suite on Ubuntu and Python 3.12. The gate is evaluated from coverage.json, not from a rounded terminal percentage.

Metric	Initial gate	Current baseline	Rationale
Statement coverage	≥ 87%	91.75% (hybrid), 89.4% (tests/ only)	Blocks a material regression without pretending 100% is achievable
Branch coverage	≥ 82%	85.01% (hybrid), 84.2% (tests/ only)	Branch coverage is the weaker signal
Combined coverage	reported	89.88% (hybrid)	Reported in the dashboard for the report
RTM references	all existing	15/16 requirements have a collected test; 14/16 verified manually	A renamed or deleted test must not silently break traceability
Tests	zero failures/errors	1,502 passed + 5 xfailed (hybrid)	Strict xfails remain visible
Mutation	evidence, not a hard gate	77.4% v2 / 81.3% covered (57.8% v1)	Scope-sensitive; guard against regression instead of an arbitrary global number
Security	evidence, not a hard gate initially	not previously measured	Findings feed Track B and the final report

Current evidence (Week 11 v2)

The authoritative mutation report is `week11/mutation_v2/mutation_v2_report.md` (PR #4). This CI document does not duplicate it; it records how the evidence is produced and consumed.

Metric	Week 7 v1 (AI-only)	Week 11 v2 (final hybrid)
Mutants	5,389	5,389
Mutation score	57.8%	77.4% (+19.6 pp)
Covered-code score	62.2%	81.3%
Killed / timeout / survived / no-tests	3,087 / 26 / 1,890 / 386	4,136 / 34 / 956 / 263
Suite	388 AI-only tests	1,507 tests (1,502 passed, 5 xfailed)

The v2 run measures the shipped hybrid suite, not only the group's extra 54 tests, and v1 ran on an earlier commit. The report states this limitation explicitly. `ci/build_dashboard.py` uses `week11/mutation_v2/mutation_v2_summary.json` as the committed fallback when a fresh CI mutation artifact is not present.

If the combined suite shifts the baseline, update the thresholds in `ci/check_coverage.py` call in `ci.yml` and record the reason in the Week 10/12 metrics log. Do not lower a gate silently to make a red run green.

Week 7 risk-based test order

`tests_ai` was ranked in Week 7 by complexity-weighted, two-year churn risk. The result is `week7/priority_order.csv` (149 AI tests, APFD 0.7722 versus 0.6369 for collection order). `ci/priority_plugin.py` reorders the hybrid suite so those high-risk tests run first; unranked tests keep their normal order. No test is removed. The first 50 executed node ids, the number of matched tests and the APFD are written to `reports/priority_order.json` and uploaded in the hybrid artifact.

To compare with collection order:

```
poetry run python -m pytest -q --no-priority-order
```

Reproducibility

Tool	Pin	Reason
Poetry	2.5.1	The version that generated `poetry.lock`
pytest	7.4.4 (Poetry lock)	Keeps Python 3.9 compatibility
pytest-bdd	9.0.0 (Python ≥3.10)	Parses Gherkin docstrings; Python 3.9 unit job skips BDD
hypothesis	6.129.5 (Poetry lock)	Track D property tests
mutmut	3.8.0 in `ci/mutation-requirements.txt`	Matches the Week 11 mutation run; requires Python ≥3.10
Python	3.9–3.14 matrix; 3.12 canonical	Matches `requires-python` and the Week 10/11 environment
Coverage	branch mode in `.coveragerc` and `--cov-branch`	Statement and branch evidence

The default dev group keeps `mutmut < 3` so the Python 3.9 unit job can install the project. The mutation job creates `.mutation-venv` and installs the pinned Week 11 tool environment separately. The mutation scope and the one deselected recursion test are documented in `week11/mutation_v2/README.md`.

Local reproduction:

```
git submodule update --init --recursive
poetry install
poetry run python -m pytest -q -p ci.priority_plugin
poetry run python ci/check_coverage.py --coverage reports/coverage.json
```

Security evidence

The `security` job is deliberately evidence-first: `pip-audit` and `bandit` write reports and the job does not yet block merges. This avoids turning an unrelated advisory in a development dependency into a red build during the project. For the final submission, summarise the findings under Track B and raise the job to blocking once high/critical findings are triaged.

Branch protection / ruleset

Create a ruleset on `master` with these required checks:

- `Quality / lint and types`
- `Unit / ubuntu-latest / 3.12`
- `Hybrid suite / coverage`

- `RTM validation`

Also enable **Require branches to be up to date before merging** and **Do not allow bypassing the above settings**. PRs from other forks may need maintainer approval before their workflows run; use a branch in the main fork for the live demonstration.

Mapping to the COMP 441 brief

Brief milestone	Deliverable	CI evidence / job
Week 5	Charter, test plan, risks, estimation, toolchain proof-of-setup	This document, `ci.yml`, pinned tool table, first green dispatch run
Week 6	AI baseline report, coverage before manual tests, RTM ≥10 requirements	`ai-baseline` job (AI-only coverage + JUnit), `rtm` job (all 16 requirements checked)
Week 7	Mutation score of AI-only suite, survivors, AI/risk-based priority order	`week7/` evidence, `mutation` nightly artifact, `priority_plugin` + `priority_order.json`, dashboard APFD
Week 8	Specialised checkpoint, TDD log, first BDD scenario	`hybrid` runs `tests_tdd/` and `tests_bdd/`; JUnit/doc evidence
Week 9	Exploratory report, hybrid suite v1, integration/acceptance suites, light performance/usability	`hybrid` runs `tests_integration/` + `tests_bdd/`; `performance` job and artifact
Week 10	Track C checkpoint, quality metrics dashboard (defect density, coverage, mutation trend)	`dashboard` artifact: coverage, test counts, defect density, mutation, Week 10 baseline comparison
Week 11	Mutation report v2, Track E checkpoint, completed TDD log	`week11/mutation_v2/mutation_v2_report.md` (77.4% v2), `tests_tdd/`, `tests_property/`, `mutation` job/artifact
Week 12	Final report, hybrid suite, RTM, CI/CD configuration, declaration form; Track B security	`ci.yml`, `dashboard`, `rtm-evidence`, `security-evidence`, `release` build/verify
Week 12 live demo	CI/CD pipeline demonstration and Q&A	`docs/ci-cd-demo.md`

Retired workflow

`.github/workflows/tests.yml` was the upstream unit-only workflow. It is replaced by the `unit` and `hybrid` jobs in `ci.yml`. The upstream poetry/poetry-core compatibility check remains in `integration.yml` and now runs nightly or on demand; it is not the project's integration suite.

Source: docs\ci-cd-demo.md

10-minute live CI/CD demonstration

Audience: COMP 441 assessors and Q&A panel
Goal: prove that the pipeline runs, that it gates the hybrid suite, and that the report evidence is downloadable rather than "a green tick".

Before the session

Run this checklist 30–60 minutes before the demo.

```
# 1. Confirm the fork's workflows are enabled and a green run exists.
gh run list --workflow=ci.yml --limit 5

# 2. Produce a fresh run without needing a commit.
gh workflow run ci.yml --ref master \
  -f run_mutation=none \
  -f run_performance=false

# 3. Wait for it and note the run id.
gh run watch
gh run list --workflow=ci.yml --limit 1

# 4. Download the evidence once, so the demo does not depend on live downloads.
mkdir -p demo-artifacts
gh run download <run-id> -n dashboard -D demo-artifacts/dashboard
gh run download <run-id> -n hybrid-evidence -D demo-artifacts/hybrid
gh run download <run-id> -n rtm-evidence -D demo-artifacts/rtm
gh run download <run-id> -n mutation-evidence -D demo-artifacts/mutation

# 5. If the latest scheduled mutation run is older, note its run id for the demo.
gh run list --workflow=ci.yml --event=schedule --limit 3
```

Also prepare:

- One open PR from a branch in the main fork (not another user's fork) so checks run without first-time-contributor approval.
- A browser tab on the repository **Actions** page.
- A terminal in the repository root.
- A fallback screen recording of the green run and artifact download.
- Optional: a throwaway draft PR with one deliberately failing test for the red-gate segment. Close it immediately after the demo.

Do **not** run the full mutation job live: it takes 20–30 minutes. Show the artifact from the latest scheduled or manually dispatched mutation run.

Timeline

0:00–0:45 — What the pipeline is

Say:

> *This is one pipeline, not a collection of scripts. It runs the hybrid suite, enforces coverage and traceability gates, and publishes every number used in the report as an artifact. The expensive mutation and upstream-compatibility jobs run nightly or on demand.*

Show the job graph in `.github/workflows/ci.yml` or the Mermaid diagram in `docs/ci-cd.md`.

0:45–2:00 — Actions tab and green run

```
gh run list --workflow=ci.yml --limit 5
gh run view <run-id>
```

Point out:

- `quality`, `unit`, `ai-baseline`, `hybrid`, `rtm`, `security`, `dashboard`.
- The unit matrix across Ubuntu/Windows/macOS and Python 3.9/3.12/3.14.
- The full mutation job is absent from this PR run by design; it is scheduled.

2:00–3:15 — Pull request checks

```
gh pr checks <pr-number>
```

Open the PR in the browser and show the checks list. Explain that `hybrid` runs the Week 7 risk order and that coverage/RTM are required checks. If the draft failing PR is available, show the red check and the blocking merge button, then switch back to the green PR. This is the "gate" part of CI/CD, not just a test runner.

3:15–4:30 — Artifacts: coverage HTML and JUnit

```
gh run download <run-id> -n hybrid-evidence -D demo-artifacts/hybrid
start demo-artifacts/hybrid/htmlcov/index.html   # macOS: open ...
```

Show:

- `coverage.xml` for tooling and `coverage.json` used by the gate script.
- `htmlcov/index.html` for line-by-line evidence.
- `junit.xml` for the machine-readable test result.
- `coverage_gate.json` with the statement and branch thresholds and `passed`.

Say:

> *The report does not rely on a screenshot of a terminal. The HTML coverage report, JUnit XML and gate JSON are attached to the run and can be downloaded by any reviewer.*

4:30–5:30 — Dashboard

```
gh run download <run-id> -n dashboard -D demo-artifacts/dashboard
type demo-artifacts\dashboard\dashboard.md   # macOS/Linux: cat ...
```

Show the run summary page too. Point out:

- Test counts and xfails.
- Statement/branch coverage and the weakest modules.
- RTM references and status counts.
- Week 10 baseline comparison.
- Mutation status (from the last mutation artifact).

- Defect density from `reports/defects.csv`.

5:30–6:30 — Week 7 priority order in action

```
gh run download <run-id> -n hybrid-evidence -D demo-artifacts/hybrid
python -m json.tool demo-artifacts/hybrid/priority_order.json
```

Point out `matched`, `collected`, `apfd_risk_based`, and the first 50 node ids. Say:

> *The Week 7 ranking was not just a report table. The same `priority_order.csv` now drives execution order in CI, so the highest-risk tests run first. Every test still runs; only the order changes. The unranked tests keep a stable order.*

Optional comparison:

```
poetry run python -m pytest -q --no-priority-order
```

6:30–7:30 — RTM validation

```
gh run download <run-id> -n rtm-evidence -D demo-artifacts/rtm
type demo-artifacts\rtm\rtm\rtm_validation.md
```

Show:

- All 16 requirement IDs present.
- Every referenced test resolves to a collected node id.
- The status distribution and the two known incomplete requirements (R15 and R16).
- The collected test list used to validate the references.

Say:

> *Renaming or deleting a test now fails CI instead of silently leaving the RTM stale.*

7:30–8:30 — Mutation evidence

```
gh run list --workflow=ci.yml --event=schedule --limit 3
gh run download <mutation-run-id> -n mutation-evidence -D demo-artifacts/mutation
type demo-artifacts\mutation\mutation_summary.json
```

Show total, killed, survived, score and the survivor list. Explain:

- Mutation is nightly/dispatch because it takes 20–30 minutes.
- The Week 11 final run uses the full hybrid suite and mutmut 3.8.0.
- `ci/mutation_summary.py` turns raw `mutmut results` into the same JSON shape consumed by the dashboard.
- The authoritative Week 11 report is `week11/mutation_v2/mutation_v2_report.md` (PR #4): **77.4%** v2 versus **57.8%** v1 (+19.6 pp), covered-code score **81.3%**. Show `week11/mutation_v2/v1_vs_v2.png` or `v1_vs_v2.csv` for the module-level gains.

8:30–9:15 — Security and release/CD

Show the `security-evidence` artifact (`pip-audit.json`, `bandit.txt`) and the `Release` workflow. Explain:

- Security is evidence-first today; high/critical findings are triaged before it becomes blocking.
- CD for a library is verified build + distribution artifact + GitHub release, not a server deployment.
- PyPI publishing is guarded to the upstream repository; the student fork does not attempt to publish.

9:15–10:00 — Close and Q&A

One-slide summary:

- One pipeline, tiered by cost.
- Hybrid suite + coverage + RTM are required.
- Mutation, performance, security and upstream compatibility are evidence jobs.
- Every report number is downloadable.
- Week 7 priority order is executable, not just documentation.

Likely questions and short answers:

Question	Answer
Why not run mutation on every PR?	20–30 minutes and 5,389 mutants; it would dominate feedback time. It runs nightly and on demand.
Why is coverage not 100%?	The gate is a ratchet near the measured baseline; 100% would reward trivial tests. Weak modules are visible in the dashboard.

Question	Answer
Why is security non-blocking?	Advisory noise in dev dependencies; findings are collected and triaged. The job can be promoted once high/critical findings are clean.
What happens if Actions is disabled on the fork?	The Actions tab banner enables workflows; `workflow_dispatch` then produces a run without a new commit.
Does the priority plugin skip tests?	No. It only reorders; `collected` equals the full hybrid suite.

Fallback if the live run is unavailable

Use the downloaded artifacts and the screen recording:

```
gh run list --workflow=ci.yml --limit 10
gh run view <last-green-run-id>
```

Then open `demo-artifacts/dashboard/dashboard.md`, `demo-artifacts/hybrid/htmlcov/index.html` and `demo-artifacts/rtm/rtm_validation.md` locally. The technical content is the same; do not spend demo time debugging the network.

CI workflow definition

Source: .github\workflows\ci.yml

```
name: COMP 441 CI

on:
  pull_request:
    branches:
      - "**"
  push:
    branches:
      - master
  workflow_dispatch:
    inputs:
      run_mutation:
        description: "Run the full mutation suite (20-30 minutes)"
        type: choice
        options:
          - none
          - full
        default: none
      run_performance:
        description: "Run the light-touch performance check"
        type: boolean
        default: false
  schedule:
    - cron: "0 3 * * 1"

permissions:
  contents: read

concurrency:
  group: comp441-ci-${{ github.workflow }}-${{ github.ref }}
  cancel-in-progress: ${{ github.event_name == 'pull_request' }}

env:
  POETRY_VERSION: "2.5.1"
  PYTHON_VERSION: "3.12"
  PIP_DISABLE_PIP_VERSION_CHECK: "1"

jobs:
  quality:
    name: Quality / lint and types
    runs-on: ubuntu-latest
    timeout-minutes: 15
    steps:
      - name: Checkout
        uses: actions/checkout@v6
        with:
          submodules: "recursive"

      - name: Set up Python
        uses: actions/setup-python@v6
        with:
          python-version: ${{ env.PYTHON_VERSION }}

      - name: Install Poetry
        run: pipx install poetry==${{ env.POETRY_VERSION }}
```

```
      - name: Configure Poetry
        shell: bash
        run: |
          poetry config virtualenvs.in-project true
          poetry env use python

      - name: Cache virtualenv
        uses: actions/cache@v5
        with:
          path: .venv
          key: venv-${{ runner.os }}-${{ env.PYTHON_VERSION }}-${{ hashFiles('**/poetry.lock') }}

      - name: Install dependencies
        shell: bash
        run: poetry install --no-interaction

      - name: Ruff check (CI scripts, blocking)
        shell: bash
        run: poetry run ruff check ci conftest.py

      - name: Ruff format check (CI scripts, blocking)
        shell: bash
        run: poetry run ruff format --check ci conftest.py

      - name: Ruff check (project baseline, evidence)
        continue-on-error: true
        shell: bash
        run: |
          mkdir -p reports/quality
          poetry run ruff check tomlkit tests tests_ai tests_manual tests_bdd \
            tests_integration tests_property tests_tdd \
            --output-format=concise 2>&1 | tee reports/quality/ruff.txt

      - name: Ruff format check (project baseline, evidence)
        continue-on-error: true
        shell: bash
        run: |
          mkdir -p reports/quality
          poetry run ruff format --check tomlkit tests tests_ai tests_manual \
            tests_bdd tests_integration tests_property tests_tdd \
            2>&1 | tee reports/quality/ruff-format.txt

      - name: Mypy (project baseline, evidence)
        continue-on-error: true
        shell: bash
        run: |
          mkdir -p reports/quality
          poetry run mypy tomlkit 2>&1 | tee reports/quality/mypy.txt

      - name: Upload quality evidence
        if: always()
        uses: actions/upload-artifact@v4
        with:
          name: quality-evidence
          path: reports/quality
          if-no-files-found: ignore

  unit:
    name: Unit / ${{ matrix.os }} / ${{ matrix.python }}
    runs-on: ${{ matrix.os }}
    timeout-minutes: 20
    strategy:
      fail-fast: false
      matrix:
        os: [ubuntu-latest, windows-latest, macos-latest]
        python: ["3.9", "3.12", "3.14"]
    steps:
      - name: Checkout
        uses: actions/checkout@v6
        with:
          submodules: "recursive"

      - name: Set up Python
        uses: actions/setup-python@v6
        with:
          python-version: ${{ matrix.python }}
          allow-prereleases: true

      - name: Get full Python version
```

```
        id: full-python-version
        shell: bash
        run: |
          echo version=$(python -c "import sys; print('-'.join(map(str, sys.version_info)))") >> "$GITHUB_OUTPUT"

      - name: Install Poetry
        run: pipx install poetry==${{ env.POETRY_VERSION }}

      - name: Configure Poetry
        shell: bash
        run: |
          poetry config virtualenvs.in-project true
          poetry env use python

      - name: Cache virtualenv
        uses: actions/cache@v5
        with:
          path: .venv
          key: venv-${{ runner.os }}-${{ steps.full-python-version.outputs.version }}-${{ hashFiles('**/poetry.lock') }}

      - name: Ensure cache is healthy
        if: steps.cache.outputs.cache-hit == 'true'
        shell: bash
        run: timeout 10s poetry run pip --version || rm -rf .venv
```

Showing the first 160 of 597 lines. Full file: `.github\workflows\ci.yml`.

Consolidated defect log (final)

Source: reports\defects.csv

id	summary	status	severity	found_by	linked_test	commit
D1	date() accepted datetime strings	fixed	medium	AI baseline	tests_ai/test_ai_api.py::TestDateTimeFactories::test_date_rejects_datetime	a4f54db
D2	Bool.__eq__ returns NotImplemented for another Bool	open	low	AI baseline	tests_ai/test_ai_items.py::TestPickle	
D3	Inline-table duplicate raises KeyAlreadyPresent instead of ParseError	open	medium	AI baseline	tests_ai/test_ai_parser.py::TestInlineTables::test_duplicate_key_in_inline	
D5	key([]) serialises to an unparseable key	open	low	AI baseline	tests_ai/test_ai_api.py::TestKeyFactory	
D6	comment() writes unparseable TOML for a bare CR	fixed	medium	Manual mutation follow-up	tests_manual/test_kill_v1.py::test_k8_add_line_comment_rejects_any_line_break	c829a38
D7	Bare CR in multiline strings	fixed	medium	Property-based test	tests_property/test_properties.py::test_d7_multiline_string_with_bare_cr	a38a3d8
D8	Insertion at position 0 drops the space after the inserted element	open	medium	Exploratory session	tests_manual/test_kill_v1.py::test_k6_insert_at_index_zero	
M1	_CustomDict.__or__ drops existing keys because copy() returns {}	open	medium	Mutation testing	tests/test_types.py::test_custom_dict_or_keeps_existing_keys	

The green run is GitHub Actions run `40412867639`. Download artefacts with `gh run download 40412867639 -n dashboard`, replacing `dashboard` with the artifact name. The workflow file is printed above; the full file is `.github/workflows/ci.yml`.

Raw evidence inventory

The pack prints the readable report and key summaries. Large raw artefacts are kept in the repository and listed here so a marker can verify them without paging through thousands of lines. Items tagged **[AI-only]** are deliberately not printed: they are the AI baseline's raw output, not the final hybrid result.

File	Size	Tag
reports\week12\ci-evidence.md	3 KB	